Hermes Agent Plugins — Technical Documentation

Prepared for the Hermes Agent Plugin Hackathon (deadline: Nov 11, 5 PM EST). Based on the official Hermes docs as of Nov 5, 2026. Verify details against `hermes plugins --help` and the docs, since the API evolves quickly.

1. What is a plugin?

A plugin is a directory (or pip package) that extends Hermes **without modifying its core code**. It can add tools the LLM can call, hook into the agent lifecycle, add slash commands and CLI subcommands, bundle skills, and (for specialised kinds) swap out whole subsystems: model provider, memory backend, context compressor, messaging platform, image generation, Desktop UI.

Plugin vs. skill vs. MCP:

	Skill	MCP server	Plugin
What it is	Markdown instructions	External process exposing tools	Python code running inside Hermes
Can change agent behaviour at runtime (hooks)	No	No	Yes
Can add platforms, providers, memory, UI	No	No	Yes

The hackathon judges favour plugins that do **more than a prompt or a skill could**, so aim for hooks, tools with real logic, or a specialised plugin kind.

2. Plugin types

Type	Purpose	How many active
General plugin	Tools, hooks, slash commands, CLI commands, bundled skills	Any combination
Memory provider	Replace/augment built-in memory (subclass `MemoryProvider`)	One

Type	Purpose	How many active
Context engine	Replace the context compressor	One
Model provider	Declare an inference backend (`register_provider(ProviderProfile(...))`)	Many loaded, you pick one via `--provider` or `config.yaml`
Platform adapter	Gateway channels (Discord, Telegram, IRC…) via `ctx.register_platform()`	Per-platform config
Image-gen backend	`ctx.register_image_gen_provider()`	Selected in config
Desktop plugin	UI for Hermes Desktop via `desktop/plugin.js` and `@hermes/plugin-sdk`	—

Internally the PluginManager distinguishes kinds: `standalone`, `backend`, `platform`, and `exclusive` (memory providers, which have their own loader). Model providers also have their own loader (`kind: model-provider`).

Not plugins, but related: config-driven integrations (TTS, STT, MCP servers declared in `config.yaml`) and gateway event hooks (`HOOK.yaml` + `handler.py` in `~/.hermes/hooks/<name>/`).

3. Anatomy of a general plugin

```
~/.hermes/plugins/calculator/
├── plugin.yaml      # manifest
├── __init__.py      # defines register(ctx)
├── schemas.py       # tool schemas the LLM reads
├── tools.py         # handler implementations
└── skills/<name>/SKILL.md   # optional bundled skill
```

Plugins live at `~/.hermes/plugins/<name>/` (or one category level deeper). Wrong depth, missing `__init__.py` (native plugins), or being disabled are the most common reasons a plugin does not show up. There is also a portable format ("Agent Plugins v1") using a root `plugin.json` and optional `mcp.json`, with no `__init__.py` required.

Minimal example

```
import json

def register(ctx):
    schema = {
        "name": "hello_world",
        "description": "Returns a friendly greeting for the given name.",
        "parameters": {
            "type": "object",
            "properties": {"name": {"type": "string", "description": "Name to
greet"}},
            "required": ["name"],
        },
    }

    def handler(args: dict, **kwargs) -> str:
        return json.dumps({"greeting": f"Hello, {args.get('name',
'world')}!"})

    ctx.register_tool(name="hello_world", toolset="hello", schema=schema,
handler=handler)
```

Drop it in `~/.hermes/plugins/hello-world/` with a `plugin.yaml`, enable it, restart Hermes: the model can call `hello_world` straight away.

4. The manifest (`plugin.yaml`)

v1 fields

Field	Purpose
`name`, `version`, `description`, `author`	Identity
`provides_tools`, `provides_hooks`	What `register()` registers. Used for validation and listings; does not gate loading
`requires_env`	Env vars needed. Simple list, or rich form with `description`, `url`, `secret`
`capabilities`	Privileged surfaces requested for consent (e.g. `tools.override`, `llm.model_override`)

v2 fields (all optional, set `manifest_version: 2`): `api_version`, `requires_plugins` (advisory dependencies with `version_range`), `python_dependencies`, `config_schema` (typed settings: `str`, `int`, `float`, `bool`, `list`, `dict`, `secret`), `config_keys`, `license`, `homepage`, `tags`, `requires_auth`, `oauth`, `provides_locales`, `python_runtime`. Unknown fields are ignored with a warning.

For catalog submission, `requires_hermes` (a SemVer floor) must also be declared.

```
name: my-plugin
version: 1.2.0
manifest_version: 2
api_version: 1
description: What it does
requires_env:
  - name: WEATHER_API_KEY
    description: "API key for OpenWeather"
    url: "https://openweathermap.org/api"
    secret: true
python_dependencies:
  - "somepkg>=1.0,<2"
config_schema:
  api_url: {type: str, default: "", description: "Service endpoint"}
```

5. The `register(ctx)` API

Hermes imports your plugin and calls `register(ctx)` once. If it raises, the plugin is disabled and Hermes keeps running.

Method	What it does
`ctx.register_tool(name, toolset, schema, handler, check_fn=None, override=False)`	Adds an LLM-callable tool. `check_fn` returning False hides it
`ctx.register_hook(name, callback)`	Subscribe to a lifecycle event
`ctx.register_command(name, handler, description=...)`	In-session slash command (CLI and gateway). Handler gets a raw args string
`ctx.register_cli_command(...)`	Adds `hermes <plugin> <subcommand>` (argparse tree)
`ctx.register_skill(name, path)`	Ships a read-only bundled skill

Method	What it does
`ctx.register_platform(...)` / `register_memory_provider(...)` / `register_context_engine(...)` / `register_image_gen_provider(...)`	Specialised kinds
`ctx.dispatch_tool(name, args)`	Call any tool with the parent agent's approvals and context
`ctx.llm.complete(...)` / `complete_structured(...)`	One-shot completion with the user's active model and auth. Overrides need `llm.model_override`
`ctx.get_config` / `set_config`	Plugin-scoped settings
`ctx.state.get/set`	Profile-scoped runtime state (max 10 MiB)
`ctx.has_capability(id)` / `ctx.has_plugin(id)`	Probes, so you can degrade gracefully

Tool handler rules

- Signature: `def handler(args: dict, **kwargs) -> str`
- Always return a **JSON string**, including on error
- Never raise; catch and return error JSON
- Accept `**kwargs` (`task_id`, `session_id`, … are passed for forward compatibility)

6. Hooks

Hooks let a plugin observe or alter the agent loop. Callbacks receive keyword arguments; always accept `**kwargs` because payloads grow additively. A crashing callback is logged and skipped; it cannot take the agent down.

Category	Hooks	Return value matters?
Tool	`pre_tool_call`, `post_tool_call`, `transform_tool_result`, `transform_terminal_output`	`pre_tool_call` can return `{"action": "block", "message": ...}` or `{"action": "modify", "args": {...}}`; `transform_*` return a replacement string

Category	Hooks	Return value matters?
Turn	`pre_llm_call`, `post_llm_call`, `transform_llm_output`	`pre_llm_call` can inject context (string or `{"context": ...}`); `transform_llm_output` replaces the final text
API request	`pre_api_request`, `post_api_request`, `api_request_error`, `pre_auxiliary_call`, `post_auxiliary_call`	Observer
Session	`on_session_start`, `on_session_end`, `on_session_finalize`, `on_session_reset`, `on_session_delete`	Observer
Other	`post_compaction`, `subagent_start/stop`, `gateway_platform_event`, kanban hooks, `pre_gateway_dispatch`, `pre_transcription`, `pre_verify`	Mostly observer

The docs list 27 valid hook events; the exact set and payloads are in the Event Hooks and Observer Hooks pages. `post_tool_call` receives `tool_name`, `args`, `result`, `duration_ms`, `status`, and more.

Example: block a dangerous command.

```
def guard(tool_name, args, **kwargs):
    if tool_name == "terminal" and "rm -rf /" in args.get("command", ""):
        return {"action": "block", "message": "Blocked by safety plugin"}

def register(ctx):
    ctx.register_hook("pre_tool_call", guard)
```

7. Discovery, loading and opt-in

Source	Path	Notes
Bundled	`<repo>/plugins/`	Ships with Hermes
User	`~/.hermes/plugins/`	Personal plugins

Source	Path	Notes
Project	`.hermes/plugins/`	Needs `HERMES_ENABLE_PROJECT_PLUGINS=true`; not loaded by the gateway
pip	`hermes_agent.plugins` entry point	Packaged distribution
Nix	`services.hermes-agent.extraPlugins`	Declarative installs

Later sources override earlier ones on a name collision (exception: bundled memory providers win).

Plugins are opt-in. Discovery only lists them. Nothing loads until its name is in `plugins.enabled` in `~/.hermes/config.yaml`. `plugins.disabled` is a deny-list that always wins. Bundled platform and backend plugins auto-load, and activate through their own config keys.

8. Security model

- **Capabilities** are consent and audit, not a sandbox. Undeclared or unconsented capabilities are off.
- **Overriding a built-in tool** needs `override=True` **and** an operator grant `plugins.entries.<id>.allow_tool_override: true`, otherwise `PluginToolOverrideError`.
- **Credentials** are read through Hermes (`agent.secret_scope.get_secret(NAME)`), never from `.env`, `auth.json` or keychains. Subscription sign-ins go through `credentialed_provider_request` with `requires_auth`.
- **Storage:** write durable data with `plugin_data_dir("name")` or `plugin_db("name")` from `plugins.plugin_storage`, never inside the install tree (`update`/`remove` wipes it).
- **Optional deps:** `ensure_import("<extra>")` at the point of use.

9. Dev workflow

```
# Develop
mkdir -p ~/.hermes/plugins/my-plugin   # plugin.yaml + __init__.py
hermes plugins enable my-plugin
hermes plugins list

# Check
hermes plugins doctor ./my-plugin --ci            # discovery, manifest,
register(ctx), registries
hermes plugins validate ./my-plugin --install-deps # the same check catalog
CI runs

# Debug
HERMES_PLUGINS_DEBUG=1 hermes plugins list         # skip reasons +
tracebacks
hermes logs --level WARNING | grep -i plugin       # ~/.hermes/logs/agent.log
# In a session: /plugins
```

Other management commands: `hermes plugins install <name|owner/repo> [--ref <sha>] [--no-enable]`, `update`, `remove`, `disable`, `search`, `capabilities`.

For tests, the Hermes repo itself builds a `PluginManager`, points `HERMES_HOME` at a tmp dir, and calls `discover_and_load()` then `invoke_hook(...)`. You can mirror that pattern.

10. Distribution and the hackathon submission

The catalog is a human-reviewed directory. A listing is one file, `plugin-catalog/<name>.yaml`, added by PR to `NousResearch/hermes-agent`. Key fields: `repo` (public https URL), `subdir`, `sha` (full 40-char commit), `version` (quoted), `requires_hermes`, `capabilities`, `tier`.

Admission rules that matter most

1. Pin an exact 40-char SHA (no branch/tag).
2. No self-updating code or remote loaders.
3. `capabilities:` must match what the plugin actually registers.
4. Extend Hermes only via public surfaces: no monkey-patching or core overrides.
5. Secrets via Hermes, respect approvals, no auto-approve or YOLO child processes.
6. Disclose network calls, shell commands, background processes and stored credentials in the PR and README. Telemetry must be opt-in.
7. No compiled binaries or unreviewable payloads.

8. Desktop plugins may only import from `@hermes/plugin-sdk` / `react`, with no `eval`, prototype patching or DOM-poking of core UI.
9. Dependencies: use upper bounds and the oldest compatible floor.

CI runs `hermes plugins validate` at the pinned SHA, including the install security scan (`dangerous` fails; `caution` warns).

Hackathon checklist

- [ ] Public repo with plugin, README, `plugin.yaml`
- [ ] `hermes plugins validate . --install-deps` passes
- [ ] PR adding `plugin-catalog/<name>.yaml` (can submit the form as soon as the PR is open)
- [ ] Demo video under 90 s on X, tagging @NousResearch
- [ ] Submit both links via the Typeform before Nov 11, 5 PM EST

11. Idea-fit tips

For a side project at a modest level, good scopes are: a `pre_tool_call` guard/audit plugin, a `transform_tool_result` compressor or redactor, a tool wrapping an API you already use for infra work, or an observability plugin on the observer hooks. Each can be built in a day, is easy to demo in 90 seconds, and uses hooks, which a skill cannot replicate.

Sources

- Hermes docs: Build a Hermes Plugin (hermes-agent.nousresearch.com/docs/developer-guide/plugins)
- Hermes docs: Plugins (…/docs/user-guide/features/plugins)
- Hermes docs: Plugin Catalog and Catalog Submission (…/docs/user-guide/features/plugin-catalog, …/developer-guide/plugins/catalog-submission)
- Hermes docs: Observer Hooks, Model Provider and Memory Provider Plugins (…/developer-guide/)